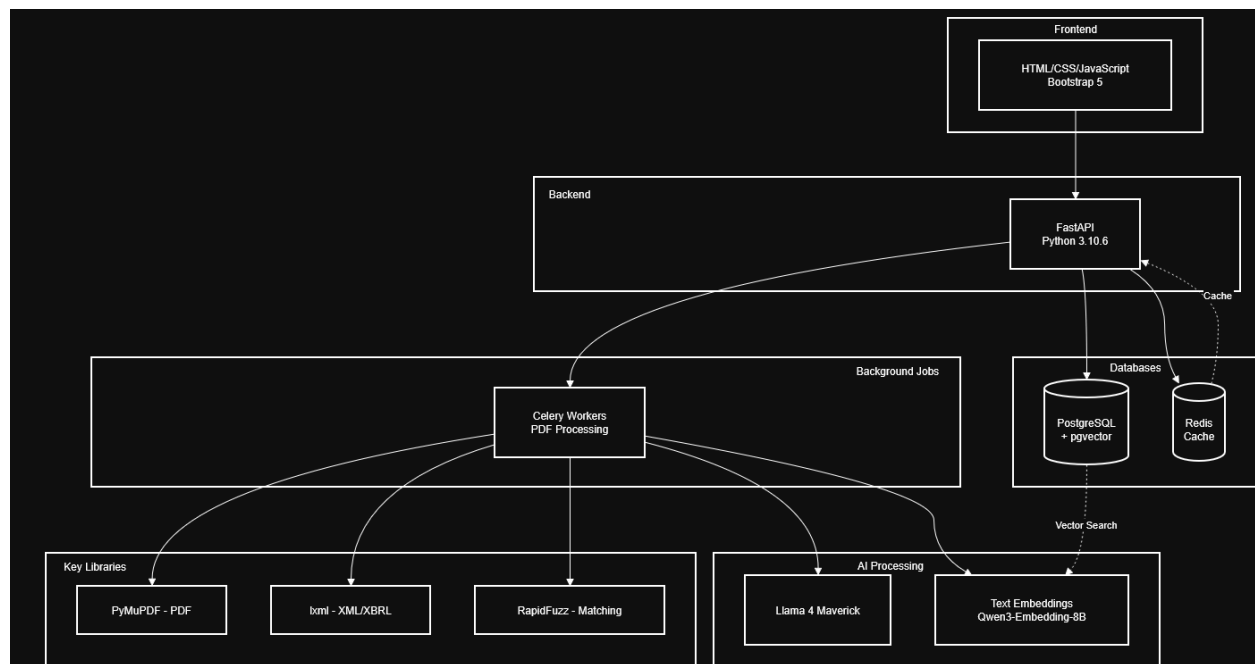


XBRL to MBRS Converter Tech Stack

XBRL to MBRS Converter Tech Stack



Backend Stack

```
### FastAPI 2.0
- Purpose: Modern async web framework with automatic API documentation
- Features:
  - Native async/await support
  - Automatic OpenAPI/Swagger documentation
  - Built-in request validation via Pydantic
- Usage: Main application framework ([main.py] (main.py))

### Uvicorn
- Purpose: Lightning-fast ASGI server
- Features:
  - Hot reload in development mode
- Configuration:
  - Host: `0.0.0.0`
  - Port: `8001`
  - Debug mode: `--reload` flag enabled in development

### Python 3.10.6
```

```
- **Required Version**: Python 3.10.6 or higher
- **Reason**: Async/await enhancements
- **Platform**: Windows
```

Database Stack

PostgreSQL

```
- **Purpose**: Primary relational database
- **Required Extensions**:
  - **pgvector**: Vector similarity search (required for semantic matching)
- **Connection**: Async via AsyncPG
- **Features**:
  - ACID transactions
  - Full-text search
  - JSON/JSONB support
  - HNSW index for vector similarity
```

SQLAlchemy 2.0

```
- **Package**: `sqlalchemy[asyncio]`
- **Purpose**: Async ORM and query builder
- **Features**:
  - Declarative models ([database.py] (database.py))
  - Async session management
  - Relationship loading strategies (eager, lazy, selectin)
  - Migration support via Alembic
```

AsyncPG

```
- **Purpose**: High-performance async PostgreSQL driver
- **Features**:
  - Native async support
  - Binary protocol (faster than psycopg2)
  - Connection pooling
- **Usage**: Backend for SQLAlchemy async engine
```

pgvector Extension

```
- **Purpose**: Vector similarity search for semantic matching
- **Vector Dimensions**: **1752 dimensions** (truncated from Qwen3-Embedding-8B)
- **Index Type**: HNSW (Hierarchical Navigable Small World)
```

```
- **Distance Metric**: Cosine similarity (`<=>` operator)
- **Usage**:
  - `MBRSTaxonomyTag.embedding` - MBRS taxonomy tag embeddings
  - `XMLTemplateField.embedding` - Template field embeddings
```

Background Processing

```
### Celery
- **Purpose**: Distributed task queue for async PDF processing
- **Broker**: Redis
- **Result Backend**: Redis
- **Queues**:
  - `pdf_processing` - PDF extraction tasks
  - `xbrl_generation` - XBRL file generation tasks
- **Configuration**:
  - Serializer: JSON
  - Task acknowledgment: Late ACK
  - Task reject on worker lost: Enabled
  - Task time limit: 3600s (1 hour)

### Celery Worker Pool (Windows-Optimized)
- **Pool Type**: `threads` (not prefork - Windows incompatible)
- **Concurrency**: 4 concurrent threads
- **Reason**: Windows doesn't support fork()
- **Startup**: `python start_celery.py` (custom Windows wrapper)
- **File**: [start_celery.py] (start\_celery.py)
```

Frontend Stack

```
### Jinja2
- **Purpose**: Server-side HTML templating
- **Features**:
  - Template inheritance
  - Macros and filters
  - Auto-escaping (XSS protection)
- **Usage**: Dashboard pages, email templates
- **Template Location**: `templates/` directory
```



```

├─ 📁 static/                                # Frontend Assets (if any)
│   └─ js/
├─ 📁 templates/                             # HTML Templates (if any)
├─ 📁 migrations/                           # Database Migrations
│   └─ README.md
├─ 📁 xbrl_mpers/                           # Sample XBRL Files (7 companies)
│   └─ [Sample company XBRL files]         # Reference implementations
├─ 📄 Data Files
│   └─ mpers_templates.json                # MPERS template structure (461KB)
├─ 📄 Configuration
│   ├── requirements.txt                   # Python dependencies
│   ├── .env                             # Environment variables (not in repo)
│   └─ .claude/settings.local.json        # Claude Code settings
└─ 📁 xbrl_venv/                            # Python Virtual Environment

```

Application Flow

```

User
├─
├─> 1. Upload PDF (/api/v1/filings/upload)
│   ├──> Validate file (size, extension)
│   ├──> Store PDF to disk (uploads/pdfs/)
│   ├──> Create FilingJob (status: PROCESSING)
│   └─> Dispatch Celery task (process_pdf_task)
│       ├──> 2. Background Processing (Celery Worker)
│       │   ├──> Stage 1: Classify pages (VLM)
│       │   │   └─> Detect statement types per page
│       │   └─> Stage 2: Extract data (VLM + templates)

```

```
└─┬> Template-guided extraction
    |
    └─> Stage 3: Semantic matching (embeddings)
        └─> Match to MBRS taxonomy tags
    |
    └─> Stage 4: Validation (linkbase)
        └─> Check calculations
    |
    └─> Update job status (REVIEW)
        |
        └─> 3. User Review (Dashboard)
            |
            └─> View extracted items
            └─> Edit/delete items
            └─> Search taxonomy tags
            └─> Confirm mappings
            └─> Mark as reviewed
                |
                └─> 4. Generate XBRL
                    |
                    └─> Validate completeness
                    └─> Generate XML (lxml)
                    └─> Save XBRL file
                    └─> Download ZIP
```


Subsequent startup and Installation

Installation

Programs to Install before setup

Programs:

1. **Visual Studio Code**

<https://code.visualstudio.com/download>

2. **Visual Studio 2022 - Community version - **Install the C++ Desktop Development Workflow****

<https://visualstudio.microsoft.com/vs/>

3. **GIT**

<https://git-scm.com/install/windows> - install the x64 not the ARM64

Install Dependencies (external programs):

1. **Postgresql - Version 17.7**

<https://www.enterprisedb.com/downloads/postgres-postgresql-downloads>

- Port: 5432
- User: postgres
- Password: aS1Ans3cR3t

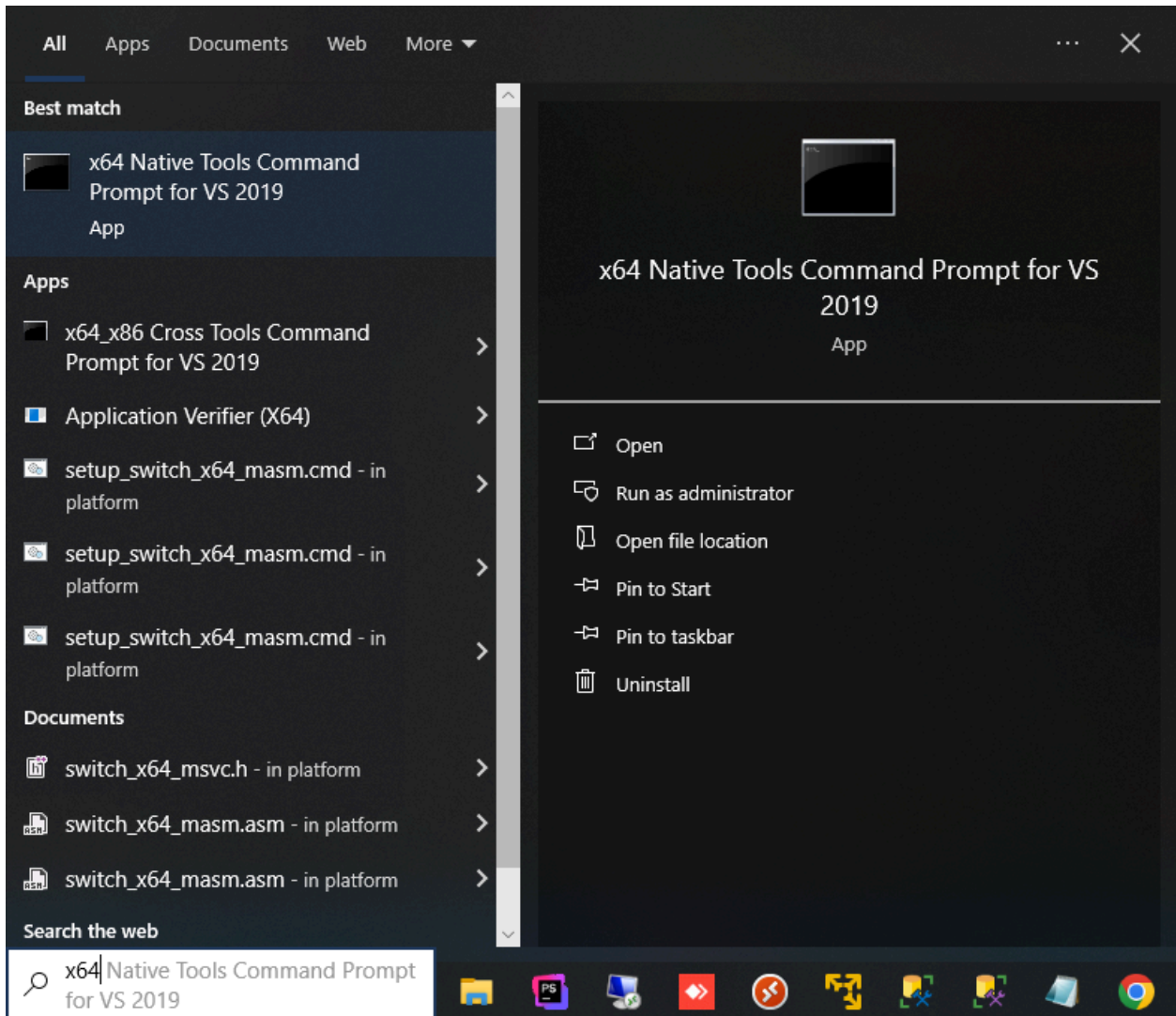
2. **Redis - Version 3.0.504**

<https://github.com/microsoftarchive/redis/releases>

- download the .msi file, this is the windows installer.
- open the .msi file, follow the instructions for installation.
- ****IMPORTANT****: Check the box that says "Add to PATH environment variable"
- Port: 6379

PGVector extension for Postgresql

1. Make sure postgresql is installed first
2. open 'x64 Native Tools Command Prompt for VS 2022' (find it in the windows search)



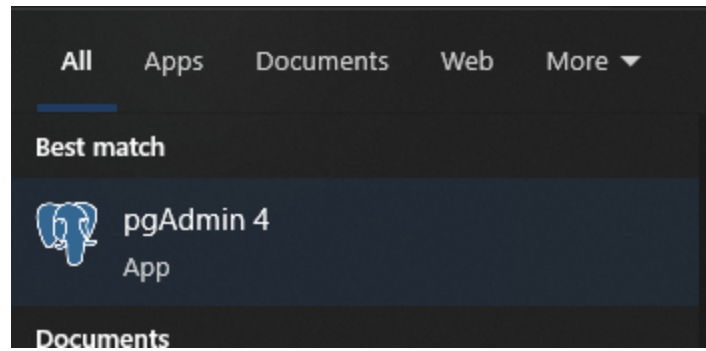
3. Run these commands (run them one at a time):

```
set "PGROOT=C:\Program Files\PostgreSQL\17"  
cd %TEMP%  
git clone --branch v0.8.0 https://github.com/pgvector/pgvector.git  
cd pgvector  
nmake /F Makefile.win  
nmake /F Makefile.win install
```

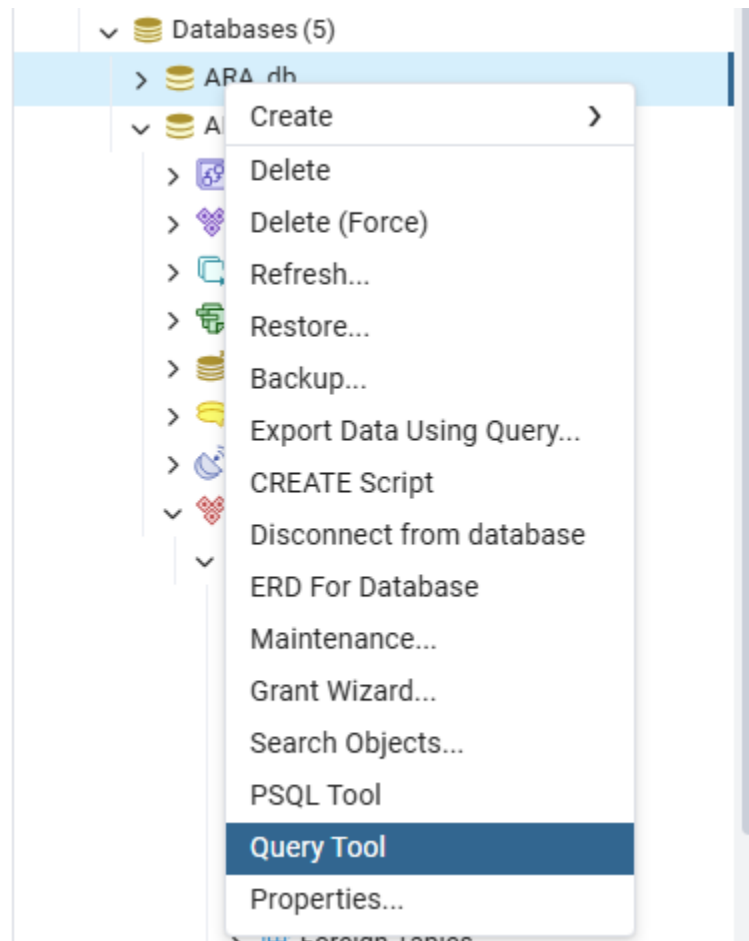
4. Run this query in pgAdmin (pgAdmin Query Tool) to activate vectors for database:

```
CREATE EXTENSION vector;
```

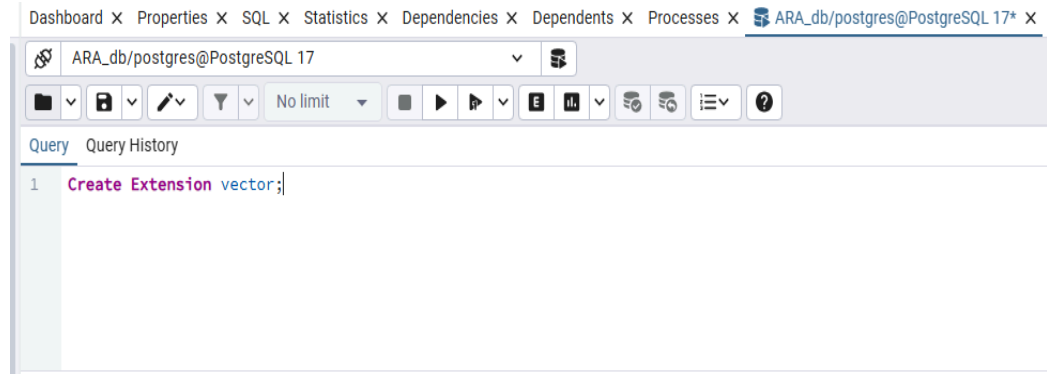
Guide to using the query tool in pgAdmin:



find pgAdmin 4 in window search



Use query tool



Enter the query to create vector.

Setup for XBRL (after installing the programs/extensions)

1. Open the XBRL folder in Visual Studio Code.
2. Create a virtual environment (venv), run this in the VS code terminal:

```
python -m venv xbrl_venv
```

3. Activate the venv (have to do this everytime if vscode is shut):

```
xbrl_venv/Scripts/activate
```

4. Install all libraries:

```
pip install -r requirements.txt
```

Subsequent/Initial Startups

1. Activate virtual environment:

```
xbrl_venv/Scripts/activate
```

2. Start redis server:

```
redis-server --port 6380
```

3. Start the celery workers:

```
python start_celery.py
```

4. Start the application:

```
python main.py
```